

Machine Learning Fundamentals

Overfitting, Underfitting & the Bias-Variance Trade-off

1. Features vs. Labels

Every machine learning model learns from data that comes in two parts. Features — also written as X — are the inputs, the things we already know about a situation. If you are trying to predict whether it will rain tomorrow, your features might be today's temperature, humidity, and cloud cover. The label — written as y — is what you want the model to figure out: in this case, rain or no rain.

Think of it like filling out a form. The fields you type into are the features. The outcome that results from those inputs is the label. The model's entire job is to learn the relationship between the two, so that when it sees a brand-new set of features it has never encountered before, it can make a smart prediction about the label.

2. Train / Validation / Test Split

We never hand all of our data to a model at once. Instead, we divide it into three separate portions. The training set is what the model actually studies — it uses this chunk to learn patterns. The validation set is kept aside during training and used to check how well different versions of the model are performing as we tune things. The test set is locked away completely and only opened once, right at the very end, to get a final honest score.

The reason we protect the test set so carefully is that it only means something if the model has never seen it. If you showed it to the model even once during development, the results would be contaminated. It would be like giving students the actual exam paper while they are still studying — their score on test day would look impressive, but you would have no idea whether they genuinely understand the subject or just memorised the answers.

3. Overfitting vs. Underfitting

Overfitting is what happens when a model learns the training data a little too well. Every real dataset contains some random noise — small flukes and quirks that are not meaningful patterns. An overfitted model picks all of those up anyway and treats them as if they matter. It performs brilliantly on the data it was trained on, but the moment it sees something new, it struggles, because the noise it memorised does not appear in fresh data.

Underfitting sits at the opposite extreme. An underfitted model is too simple to capture what is actually going on in the data. Imagine drawing a straight line through data that clearly follows a curve — no matter how you angle the line, you are going to miss most of the points. An underfitted model makes poor predictions everywhere, both on training data and new data, because it never really understood the underlying pattern. The goal is a model that sits comfortably between these two extremes: complex enough to learn the real signal, but not so complex that it chases noise.

4. The Bias-Variance Trade-off

Bias and variance are two types of error that push against each other, and finding the right balance between them is one of the central challenges in machine learning. A high-bias model has oversimplified the problem — it makes the same kind of mistake over and over because it started with assumptions that were too rigid. A high-variance model is the opposite: it is so sensitive to whatever training data it happened to see that small changes in the data cause wildly different predictions. Simple models tend to have high bias and low variance; complex models tend to have low bias but high variance. Make a model more flexible to fix bias, and variance starts to rise. Tighten the model to reduce variance, and bias creeps back up. There is no way to eliminate both at the same time. The sweet spot is wherever the combined total error is lowest — not the simplest model, not the most powerful one, but the one that generalises most reliably to data it has never seen before.

Core idea: a model that fits the training data perfectly is almost never the model that predicts best.